

SCX 框架完整模块分析

State-Conditioned eXpertise — 面向数据价值评估与专家引导
学习的状态条件专家性框架

生成日期：2026-06-25 | 版本：v0.1.0-draft

目录

1. 模块 1：状态发现与表示
 2. 模块 2：专家可靠性估计
 3. 模块 3：数据四分类
 4. 模块 4：状态数据估值
 5. 模块 5：专家路由与仲裁
 6. 模块 6：SCX-Compress（冗余压缩）
 7. 模块 7：主动学习策略
 8. 模块 8：与 Paper 1-3 的连接
 9. 完整系统架构图
-

模块 1：状态发现与表示 (State Discovery & Representation)

1.1 问题设定

给定原始输入空间 $\mathcal{X}$ (原子结构、图像、文本等) 和一个表示映射 $\phi: \mathcal{X} \rightarrow \mathbb{R}^d$, 状态发现的目标是将高维、异构的输入空间划分为 K 个语义上有意义的状态 $S = \{S_1, S_2, \dots, S_K\}$, 使得每个状态内部的样本具有相似的结构/语义属性, 而不同状态之间的样本在统计和行为上存在显著差异。

1.2 数学模型

1.2.1 表示映射 定义特征提取函数 $\phi: \mathcal{X} \rightarrow \mathbb{R}^d$, 将原始输入映射到 d 维特征空间。在科学计算场景中, 常见选择:

- **ACE (Atomic Cluster Expansion):** 将原子邻域环境展开为 d 维 bispectrum 系数, $d \sim O(N_{\max}^3)$ 其中 $N_{\max}$ 是截断阶数
- **SOAP (Smooth Overlap of Atomic Positions):** 利用高斯核构造原子环境相似性, $d \sim n_{\max} \times l_{\max}$
- **神经网络 embedding:** $h(x) = \text{Encoder}(x; \theta) \in \mathbb{R}^d$
- **CLIP embedding:** 多模态对齐表示, $d = 512$ 或 768

1.2.2 降维 原始表示 $\phi(x) \in \mathbb{R}^d$ 通常维度较高 (ACE 可达数千维), 需要降维到 $p \ll d$ 以进行有效的状态发现。

PCA (Principal Component Analysis):

找到投影矩阵 $U \in \mathbb{R}^{d \times p}$ 使得投影后的方差最大:

$$U^* = \arg \max_{U: U^\top U = I} \text{Tr}(U^\top \Sigma U)$$

其中 $\Sigma = \frac{1}{N} \sum_{i=1}^N (\phi(x_i) - \bar{\phi})(\phi(x_i) - \bar{\phi})^\top$ 是协方差矩阵。投影后的表示记为 $\tilde{\phi}(x) = U^\top \phi(x) \in \mathbb{R}^p$ 。

UMAP (Uniform Manifold Approximation and Projection):

构造加权 k -NN 图 $G = (V, E, w)$ ，其中权重 $w_{ij} = \exp(-(d_{ij} - \rho_i)/\sigma_i)$ ，然后最小化低维嵌入的交叉熵：

$$L_{\text{UMAP}} = \sum_{(i,j) \in E} \left[w_{ij} \log \left(\frac{w_{ij}}{v_{ij}} \right) + (1 - w_{ij}) \log \left(\frac{1 - w_{ij}}{1 - v_{ij}} \right) \right]$$

其中 $v_{ij} = (1 + a \cdot \|\tilde{\phi}(x_i) - \tilde{\phi}(x_j)\|^{2b})^{-1}$ 是低维空间中的相似度。

PaCMAP (Pairwise Controlled Manifold Approximation):

结合近邻保持、中程排斥和远距离排斥三项损失：

$$L_{\text{PaCMAP}} = \sum_{(i,j) \in \text{FP}} w_{ij} \cdot d_{ij}^2 + \sum_{(i,j) \in \text{MN}} \max(0, 1 - d_{ij}) + \sum_{(i,j) \in \text{FN}} \max(0, 1 - d_{ij})$$

其中 FP = 近邻对，MN = 中等排斥对，FN = 远距离排斥对。

1.2.3 状态分配 硬分配（硬聚类）:

对于每个输入 x ，分配到一个离散状态 $s(x) \in \{1, \dots, K\}$ ：

- K-means: $s(x) = \arg \min_k \|\tilde{\phi}(x) - \mu_k\|^2$
- 谱聚类: $s(x) = \arg \min_k \text{distance in spectral embedding space}$

软分配（软聚类，推荐使用）：

定义状态条件概率 $\gamma_s(x) = P(s|x)$ ，满足 $\sum_{s=1}^K \gamma_s(x) = 1$ 。

高斯混合模型 (GMM) 是最自然的选择：

$$\gamma_s(x) = \frac{\pi_s \cdot \mathcal{N}(\tilde{\phi}(x); \mu_s, \Sigma_s)}{\sum_{t=1}^K \pi_t \cdot \mathcal{N}(\tilde{\phi}(x); \mu_t, \Sigma_t)}$$

其中 $\{\pi_s, \mu_s, \Sigma_s\}_{s=1}^K$ 通过 EM 算法从数据中估计：

E-step:

$$\gamma_s(x_i) = \frac{\pi_s \cdot \mathcal{N}(\tilde{\phi}(x_i); \mu_s, \Sigma_s)}{\sum_t \pi_t \cdot \mathcal{N}(\tilde{\phi}(x_i); \mu_t, \Sigma_t)}$$

M-step:

$$N_s = \sum_i \gamma_s(x_i), \quad \pi_s = \frac{N_s}{N}, \quad \mu_s = \frac{1}{N_s} \sum_i \gamma_s(x_i) \tilde{\phi}(x_i)$$

$$\Sigma_s = \frac{1}{N_s} \sum_i \gamma_s(x_i) (\tilde{\phi}(x_i) - \mu_s)(\tilde{\phi}(x_i) - \mu_s)^\top$$

状态的后验概率（软分配的另一种解释）：

$$P(x \in S_s) = \mathbb{E}_{x \sim P(\cdot|s)}[\mathbf{1}_{S_s}(x)] \approx \frac{1}{N_s} \sum_{i=1}^N \gamma_s(x_i)$$

1.2.4 最优状态数 K 的选择

- **贝叶斯信息准则 (BIC):** $\text{BIC}(K) = -2 \log \mathcal{L}(K) + p_K \log N$ ，选择使 BIC 最小的 K
- **轮廓系数 (Silhouette Score):** $S(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$ ，其中 $a(i)$ 是类内距离， $b(i)$ 是最近类间距离

- 稳定性分析: 在不同随机初始化和 bootstrap 子集上运行聚类, 选择最稳定的 K

1.3 输入输出规范

Input:

- `X`: ndarray, shape=(N , d_X), 原始输入数据或特征
- `phi_func`: callable, $X \rightarrow R^d$, 表示映射函数
- `n_states`: int, 状态数 K (或自动选择)
- `dim_red`: str, 降维方法: 'pca'/'umap'/'pacmap'
- `n_components`: int, p , 降维目标维度

Output:

- `states`: list[State], 长度为 K 的状态对象列表
每个 State 包含:
 - `id`: int, 状态编号
 - `center`: ndarray, shape=(p ,), 状态中心 (GMM mean)
 - `cov`: ndarray, shape=(p , p), 状态协方差 (GMM cov)
 - `weight`: float, 先验概率 π_s
 - `members`: list[int], 属于该状态的样本索引
 - `gamma`: ndarray, shape=(N , K), 软分配矩阵
- `phi_X`: ndarray, shape=(N , d), 表示映射后的特征
- `phi_tilde`: ndarray, shape=(N , p), 降维后的特征
- `reducer`: object, 降维对象 (可用于 transform 新数据)
- `clusterer`: object, 聚类对象 (可用于 predict 新数据)

1.4 实现方法

算法 1: 状态发现管道

Algorithm: StateDiscovery(X , ϕ _func, n _states, method='soft')

Step 1: 表示提取

```
Phi_X = phi_func(X) # X -> R^d
```

Step 2: 降维 (可选)

```
if dim_red == 'pca':
    Phi_tilde = PCA(Phi_X, n_components=p)
elif dim_red == 'umap':
    Phi_tilde = UMAP(Phi_X, n_components=p)
elif dim_red == 'pacmap':
    Phi_tilde = PaCMAP(Phi_X, n_components=p)
```

Step 3: 状态分配

```
if method == 'soft':
    gmm = GMM(n_components=n_states)
    gamma = gmm.fit_predict_proba(Phi_tilde)
elif method == 'hard':
    kmeans = KMeans(n_clusters=n_states)
    labels = kmeans.fit_predict(Phi_tilde)
    gamma = one_hot(labels, n_states)
```

Step 4: 状态统计

```
for s in 1..K:
    rho[s] = mean(gamma[:, s]) # 状态出现概率
    for each property p:
```

```
mean_p[s] = weighted_mean(X_property, gamma[:, s])
```

Result: {states, gamma, phi_X, phi_tilde}

1.5 与其他模块的协调

下游模块	从本模块接收的数据	用途
模块 2 (可靠性估计)	$\gamma_s(x), \rho(s)$	按状态估计专家误差
模块 3 (数据四类)	$\gamma_s(x), \rho(s)$, 状态边界	计算 C(s), N(s), D(s)
模块 5 (专家路由)	$\gamma_s(x)$	计算软分配的加权路由
模块 7 (主动学习)	$\rho(s)$, 状态统计	状态级采集函数
模块 6 (压缩)	$\gamma_s(x)$, 边界分数	保留边界 anchor

1.6 计算复杂度

步骤	时间复杂度	空间复杂度
表示映射 $\phi(X)$	$O(N \cdot C_\phi)$	$O(N \cdot d)$
PCA 降维	$O(Nd^2 + d^3)$	$O(d^2)$
UMAP 降维	$O(N \log N \cdot p)$	$O(N \cdot k)$
GMM 聚类 (EM)	$O(NKp^2 \cdot T)$	$O(NK + Kp^2)$
K-means 聚类	$O(NKp \cdot T)$	$O(NK)$

其中 C_ϕ 是 ϕ 的计算成本, T 是迭代次数, k 是 UMAP 的近邻数。

1.7 关键超参数

参数	符号	默认值	含义
表示维度	d	任务依赖	特征映射输出维度
降维维度	p	2-50	降维后维度 (可视化常用 2-3, 聚类常用 10-50)
状态数	K	自动	聚类簇数, 用 BIC/轮廓系数调
聚类型别	-	soft	soft (GMM) / hard (K-means)
降维方法	-	PCA	PCA/UMAP/PaCMAP, 大样本用 UMAP
GMM 协方差类型	-	full	full/diag/tied/spherical, 影响参数数

模块 2：专家可靠性估计 (Expert Reliability Estimation)

2.1 问题设定

给定 M 个专家 $\{f_1, f_2, \dots, f_M\}$ ，每个专家是一个函数 $f_m : \mathcal{X} \rightarrow \mathcal{Y}$ 。核心问题是：对于每个专家 m 和每个状态 s ，专家在该状态下的可靠性是多少？这里的可靠性在不同标注预算场景下有不同定义。

2.2 数学模型

2.2.1 有标签场景 (Supervised) 设有标签数据集 $\mathcal{D}_L = \{(x_i, y_i)\}_{i=1}^{N_L}$ ，其中 $y_i = f^*(x_i)$ 来自 oracle（如 DFT 计算、专家标注）。

定义 1：状态条件专家风险 (State-Conditioned Expert Risk)

$$R_m(s) = \mathbb{E}_{x \sim P(\cdot|s)}[\ell(f_m(x), f^*(x))]$$

经验估计：

$$\hat{R}_m(s) = \frac{\sum_{i=1}^{N_L} \gamma_s(x_i) \cdot \ell(f_m(x_i), y_i)}{\sum_{i=1}^{N_L} \gamma_s(x_i)}$$

其中 $\gamma_s(x)$ 是来自模块 1 的软分配概率。

定义 2：SCX 可靠性 (SCX Reliability)

$$\text{SCX}_m(s) = \mathbb{P}(\ell(f_m(x), y) < \tau \mid x \in s)$$

即专家 m 在状态 s 中做出可接受预测（误差低于阈值 τ ）的概率。

经验估计：

$$\widehat{\text{SCX}}_m(s) = \frac{\sum_{i=1}^{N_L} \gamma_s(x_i) \cdot \mathbf{1}[\ell(f_m(x_i), y_i) < \tau]}{\sum_{i=1}^{N_L} \gamma_s(x_i)}$$

误差的方差估计（用于不确定性量化）：

$$\mathbb{V}[\hat{R}_m(s)] \approx \frac{1}{N_s} \cdot \frac{1}{N_s - 1} \sum_{i=1}^{N_L} \gamma_s(x_i) \left(\ell(f_m(x_i), y_i) - \hat{R}_m(s) \right)^2$$

其中 $N_s = \sum_i \gamma_s(x_i)$ 是状态 s 的有效样本量。

2.2.2 无标签场景 (Unsupervised) 当没有标签时，利用专家分歧作为代理：

$$\text{Disagreement}_m(s) = \frac{1}{M-1} \sum_{n \neq m} \mathbb{E}_{x \sim P(\cdot|s)} [\ell(f_m(x), f_n(x))]$$

经验估计：

$$\hat{D}_m(s) = \frac{1}{M-1} \sum_{n \neq m} \frac{\sum_i \gamma_s(x_i) \cdot \ell(f_m(x_i), f_n(x_i))}{\sum_i \gamma_s(x_i)}$$

无标签下的可靠性近似：

在合理的假设下（专家群体平均表现足以代表真实误差），我们可以用归一化分歧近似 SCX 可靠性：

$$\widehat{\text{SCX}}_m^{(\text{unsup})}(s) = 1 - \sigma \left(\frac{\hat{D}_m(s) - \mu_D}{\sigma_D} \right)$$

其中 $\sigma(\cdot)$ 是 sigmoid 函数， μ_D, σ_D 是所有 (m, s) 对的分歧均值和标准差。

2.2.3 小样本场景 (Few-shot) 当状态 s 的标注样本很少 ($N_s < N_{\min}$), 直接经验估计不可靠。采用以下方法:

方法 A: 贝叶斯平滑 (Bayesian Smoothing)

$$\hat{R}_m^{(\text{Bayes})}(s) = \frac{N_s \cdot \hat{R}_m(s) + \lambda \cdot \hat{R}_m(\cdot)}{N_s + \lambda}$$

其中 $\hat{R}_m(\cdot) = \frac{1}{K} \sum_{s'} \hat{R}_m(s')$ 是专家 m 的全局平均风险, λ 是平滑强度。

方法 B: 层次贝叶斯模型 (Hierarchical Bayesian)

假设各状态的风险服从共同的先验分布:

$$\begin{aligned} R_m(s) &\sim \mathcal{N}(\mu_m, \tau_m^2) \quad (\text{先验}) \\ \ell_{mi} | R_m(s) &\sim \mathcal{N}(R_m(s), \sigma_m^2/w_{si}) \quad (\text{似然}) \end{aligned}$$

其中 $w_{si} = \gamma_s(x_i)$ 是软分配权重。后验均值为:

$$\hat{R}_m^{(\text{HB})}(s) = \frac{\frac{N_s}{\sigma_m^2} \cdot \hat{R}_m(s) + \frac{1}{\tau_m^2} \cdot \mu_m}{\frac{N_s}{\sigma_m^2} + \frac{1}{\tau_m^2}}$$

方法 C: 迁移估计 (Transfer Estimation)

利用状态表示之间的相似性:

$$\hat{R}_m^{(\text{Transfer})}(s) = \frac{\sum_{s'} \kappa(\mu_s, \mu_{s'}) \cdot N_{s'} \cdot \hat{R}_m(s')}{\sum_{s'} \kappa(\mu_s, \mu_{s'}) \cdot N_{s'}}$$

其中 $\kappa(\mu_s, \mu_{s'}) = \exp(-\beta \|\mu_s - \mu_{s'}\|^2)$ 是基于状态中心相似度的核函数。

2.3 输入输出规范

Input:

- experts: list[callable], M 个专家函数 $f_m: X \rightarrow Y$
- X_labeled: ndarray, shape=(N_L, d_X), 标注数据
- y_labeled: ndarray, shape=(N_L,), 标签
- gamma: ndarray, shape=(N_L, K), 软分配 (来自模块1)
- loss_fn: callable, 损失函数 $\ell(y_{\text{pred}}, y_{\text{true}})$
- tau: float, SCX 阈值 (默认 0.1)
- method: str, 'supervised'/'unsupervised'/'bayesian'/'hierarchical'
- lambda_smooth: float, 贝叶斯平滑强度 (默认 5.0)

Output:

- R: ndarray, shape=(M, K), 状态条件专家风险矩阵
- R_var: ndarray, shape=(M, K), 风险估计的方差
- SCX: ndarray, shape=(M, K), SCX 可靠性矩阵
- disagreement: ndarray, shape=(M, K), 专家分歧 (无标签)

2.4 实现方法

算法 2: 专家可靠性估计

Algorithm: EstimateExpertReliability(L, U, gamma, experts, method)

Phase 1: 计算加权损失

for each expert m:

for each sample i:

loss_mi = loss_fn($f_m(x_i)$, y_i) # 有标签

OR dis_mi = mean_{n!=m} loss($f_m(x_i)$, $f_n(x_i)$) # 无标签

Phase 2: 按状态聚合

```

for each state s:
    N_s = sum_i gamma[i, s] + eps
    for each expert m:
        R[m, s] = sum_i gamma[i, s] * loss_mi / N_s
        SCX[m, s] = sum_i gamma[i, s] * I[loss_mi < tau] / N_s

```

Phase 3: 小样本校正 (若 $N_s < N_{\min}$)

```

for each state s where  $N_s < N_{\min}$ :
    for each expert m:
        R[m, s] = bayesian_smooth(R[m, s], R[m, :], N_s, lambda)

```

Phase 4: 不确定性量化

```

R_var[m, s] = compute_variance(loss_mi, gamma[:, s], N_s)

```

Result: {R, R_var, SCX}

2.5 与其他模块的协调

数据消费者	接收的数据	用途
模块 3 (四分类)	$R_m(s), SCX_m(s)$	计算 $C(s)$, ExpertConflict
模块 4 (估值)	$R_m(s), SCX_m(s)$	计算 $\max_m$ $SCX_m(s)$
模块 5 (路由)	$R_m(s), SCX_m(s)$	硬路由和软加 权

数据消费者	接收的数据	用途
模块 7 (主动学习)	$\max_m \text{SCX}_m(s)$	route 动作的 效用计算

2.6 计算复杂度

步骤	时间复杂度	空间复杂度
计算所有专家损失	$O(M \cdot N \cdot C_f)$	$O(M \cdot N)$
状态聚合	$O(M \cdot N \cdot K)$	$O(M \cdot K)$
贝叶斯平滑	$O(M \cdot K)$	$O(M \cdot K)$
无标签分歧	$O(M^2 \cdot N \cdot C_f)$	$O(M \cdot N)$

其中 C_f 是单次专家推理的成本。

2.7 关键超参数

参数	符号	默认值	含义
SCX 阈值	τ	0.1	误差容限, 定义”可接受预测”
平滑强度	λ	5.0	小样本向全局均值收缩的程度
最小有效样本	$N_{\min}$	10	低于此值触发小样本校正
迁移核带宽	β	1.0	状态间相似度核的带宽
层次先验方差	τ_m^2	1.0	专家风险跨状态变化的先验方差

模块 3：数据四分类 (Data Four-Classification)

3.1 问题设定

数据分类不是对原始数据标签的分类，而是对每个状态 s 的信息类型分类。目标是判断每个状态属于以下四类中的哪一类：**Valuable**（值得标注）、**Redundant**（冗余）、**Noisy**（噪声）、**Expert-dependent**（依赖特定专家）。这个分类直接驱动后续的决策（标注、压缩、路由等）。

3.2 数学模型

3.2.1 可学习性分数 (Learnability Score)

$$L(s) = C(s) \cdot [1 - N(s)]$$

其中 $C(s) \in [0, 1]$ 是一致性 (Consistency)， $N(s) \in [0, 1]$ 是噪声分数 (Noise Score)。

- $L(s) \rightarrow 1$: 状态高度可学习（一致且无噪声）
- $L(s) \rightarrow 0$: 状态不可学习（不一致或有噪声）

3.2.2 一致性度量 (Consistency Measures) 一致性 $C(s)$ 有多种定义方式，适用于不同场景：

度量 A：标签一致性 (Label Consistency)

当有多个标注者或多次标注时：

$$C_{\text{label}}(s) = \frac{1}{\binom{M}{2}} \sum_{m < n} \mathbb{E}_{x \sim P(\cdot|s)} [\mathbf{1}[f_m(x) = f_n(x)]]$$

当只有单一 oracle 标注时，用状态内方差：

$$C_{\text{var}}(s) = \exp\left(-\frac{\text{Var}(y|s)}{\sigma_0^2}\right), \quad \text{Var}(y|s) = \frac{\sum_i \gamma_s(x_i)(y_i - \bar{y}_s)^2}{\sum_i \gamma_s(x_i)}$$

度量 B：专家预测一致性 (Expert Agreement Consistency)

利用专家的预测分歧：

$$C_{\text{expert}}(s) = 1 - \frac{1}{M} \sum_{m=1}^M \frac{\hat{D}_m(s)}{\max_{m'} \hat{D}_{m'}(s) + \varepsilon}$$

其中 $\hat{D}_m(s)$ 是专家 m 在状态 s 的平均分歧。

度量 C：密度邻域一致性 (Density Neighborhood Consistency)

在无标签情况下，假设附近样本应该具有相似的标签/预测：

$$C_{\text{density}}(s) = \frac{1}{|s|} \sum_{x_i \in s} \frac{1}{k} \sum_{x_j \in \text{NN}_k(x_i)} \mathbf{1}[\hat{y}_i \approx \hat{y}_j]$$

其中 $\hat{y}_i$ 是某个基模型的伪标签。

综合一致性（推荐组合方式）：

$$C(s) = \alpha \cdot C_{\text{label}}(s) + \beta \cdot C_{\text{expert}}(s) + \gamma \cdot C_{\text{density}}(s)$$

其中 $\alpha + \beta + \gamma = 1$ ，根据标注预算调整。

3.2.3 噪声分数 (Noise Score) 逐样本噪声分数:

$$\text{NoiseScore}(x_i) = r_i \cdot \frac{1}{\rho(s_i) + \varepsilon} \cdot [1 - C(s_i)]$$

其中 $r_i = \ell(f(x_i), y_i)$ 是当前模型的预测误差。

分解分析: - 高误差 + 高密度 + 高一致性 $\rightarrow \text{NoiseScore} \rightarrow 0 \rightarrow$ 可学习困难样本
- 高误差 + 低密度 + 低一致性 $\rightarrow \text{NoiseScore} \rightarrow \text{large} \rightarrow$ 疑似噪声
- 低误差 $\rightarrow \text{NoiseScore} \rightarrow 0 \rightarrow$ 已正确预测

状态级噪声分数:

$$N(s) = \sigma \left(\frac{1}{|s|} \sum_{x_i \in s} \text{NoiseScore}(x_i) \right)$$

其中 $\sigma(\cdot)$ 是 sigmoid 归一化函数, 确保 $N(s) \in [0, 1]$ 。

3.2.4 冗余度估计 (Redundancy Estimation) 定义: 冗余度 $D(s) \in [0, 1]$ 衡量状态 s 在当前训练集中被充分覆盖的程度。

方法 A: 基于样本量

$$D_{\text{count}}(s) = 1 - \exp \left(-\frac{|\mathcal{D}_{\text{train}} \cap s|}{n_{\text{eff}}(s)} \right)$$

其中 $n_{\text{eff}}(s)$ 是覆盖状态 s 所需的有效样本量, 与状态复杂度成正比。

方法 B: 基于多样性

$$D_{\text{div}}(s) = 1 - \frac{H(\mathcal{D}_{\text{train}} \cap s)}{H(\mathcal{U} \cap s)}$$

其中 $H(\cdot)$ 是多样性的度量（如特征空间中的覆盖体积分数）， $\mathcal{U}$ 是未标注池。

方法 C：基于模型不确定性

$$D_{\text{model}}(s) = \exp \left(-\frac{1}{|s|} \sum_{x_i \in s} \mathbb{H}[p(y|x_i; \theta)] \right)$$

其中 $\mathbb{H}[\cdot]$ 是预测熵。当模型对状态内所有样本都确定时， $D(s) \rightarrow 1$ 。

最终冗余度（综合）：

$$D(s) = \min(1, D_{\text{count}}(s) + D_{\text{model}}(s))$$

3.2.5 边界检测 (Boundary Detection) 状态边界的样本具有特殊的价值——它们定义状态边界，且对路由决策至关重要。

边界分数：

$$\text{BoundaryScore}(x) = 1 - (\gamma_{(1)}(x) - \gamma_{(2)}(x))$$

其中 $\gamma_{(1)}(x)$ 和 $\gamma_{(2)}(x)$ 是最大的两个软分配概率。

- $\text{BoundaryScore} \rightarrow 0$ ：明确属于某个状态
- $\text{BoundaryScore} \rightarrow 1$ ：在状态边界上，分配模糊

边界状态标记：

$$s_{\text{boundary}} = \{x \mid \text{BoundaryScore}(x) > \tau_{\text{boundary}}\}$$

这些样本在后续的压缩（模块 6）和主动学习（模块 7）中享有特殊待遇。

3.2.6 四分类规则

类别	判定条件	符号条件
Valuable	高误差 + 高密度 + 高可 学习性 + 低冗余	$\bar{r}(s) > \tau_r, \rho(s) > \tau_\rho,$ $L(s) > \tau_L, D(s) < \tau_D$
Redundant	低误差 + 高冗余	$\bar{r}(s) < \tau_r, D(s) > \tau_D$
Noisy	高误差 + 低密度 + 低一 致性	$\bar{r}(s) > \tau_r, \rho(s) < \tau_\rho,$ $C(s) < \tau_C$
Expert- dependent	高专家分歧 + 某专家明显 优于其他	$\max_m \text{SCX}_m(s) \gg$ $\text{SCX}_{\text{others}}(s)$

若同时满足多个条件，优先级：Noisy > Expert-dependent > Valuable > Redundant。

3.3 输入输出规范

Input:

- X: ndarray, shape=(N, d_X), 所有输入数据
- y: ndarray, shape=(N,), 标签（或 None）
- gamma: ndarray, shape=(N, K), 软分配（模块1）
- R: ndarray, shape=(M, K), 专家风险（模块2）
- SCX: ndarray, shape=(M, K), SCX可靠性（模块2）
- experts: list[callable], M 个专家
- model: callable, 当前模型 $f: X \rightarrow Y$
- params: dict, 阈值参数

Output:

- classification: dict[state_id -> Category]
 每个状态映射到 {VALUABLE, REDUNDANT, NOISY, EXPERT_DEPENDENT}
- L: ndarray, shape=(K,), 可学习性分数
- C: ndarray, shape=(K,), 一致性分数
- N: ndarray, shape=(K,), 噪声分数
- D: ndarray, shape=(K,), 冗余度
- r_bar: ndarray, shape=(K,), 状态平均误差
- rho: ndarray, shape=(K,), 状态概率
- boundary_idx: list[int], 边界样本索引
- NoiseScore: ndarray, shape=(N,), 逐样本噪声分数

3.4 实现方法

算法 3: 数据四分类

Algorithm: FourClassify(gamma, R, SCX, X, y, params)

Step 1: 计算状态统计量

```

for s in 1..K:
    r_bar[s] = weighted_mean(r_i, gamma[:,s])    # 状态平均误差
    rho[s] = mean(gamma[:,s])                    # 状态出现概率

```

Step 2: 计算一致性 C(s)

```

if labeled data available:
    C_label[s] = intra_state_variance(y, gamma[:,s])
else:
    C_expert[s] = 1 - normalized_disagreement(R[:,s])
C[s] = combine(C_label[s], C_expert[s])

```

Step 3: 计算噪声分数 $N(s)$

```
for i in 1..N:
    s_i = argmax_s gamma[i,:]
    NoiseScore[i] = r_i * (1 / (rho[s_i] + eps)) * (1 - C[s_i])
N[s] = sigmoid(mean(NoiseScore[members_of_s]))
```

Step 4: 计算可学习性 $L(s)$

```
L[s] = C[s] * (1 - N[s])
```

Step 5: 计算冗余度 $D(s)$

```
D_count[s] = 1 - exp(-n_train[s] / n_eff[s])
D[s] = min(1, D_count[s])
```

Step 6: 边界检测

```
for i in 1..N:
    gamma_sorted = sort(gamma[i,:], descending)
    boundary_score[i] = 1 - (gamma_sorted[0] - gamma_sorted[1])
boundary_idx = where(boundary_score > tau_boundary)
```

Step 7: 分类状态

```
for s in 1..K:
    if r_bar[s] > tau_r and rho[s] < tau_rho and C[s] < tau_C:
        class[s] = NOISY
    elif max(SCX[:,s]) - second_max(SCX[:,s]) > tau_SCX_gap:
        class[s] = EXPERT_DEPENDENT
    elif r_bar[s] > tau_r and rho[s] > tau_rho and L[s] > tau_L and D[s] < tau_D:
        class[s] = VALUABLE
    elif r_bar[s] < tau_r and D[s] > tau_D:
```

```

        class[s] = REDUNDANT
    else:
        class[s] = VALUABLE # default to valuable (conservative)

```

Result: {classification, L, C, N, D, boundary_idx}

3.5 边界情况处理

1. **空状态 (Empty State):** 若 $\rho(s) \approx 0$, 标记为 REDUNDANT (无数据可贡献)
2. **新状态 (Novel State):** 若 $N_s < N_{\min}$, 标记为 VALUABLE (默认值得探索)
3. **高冲突状态 (High Conflict):** 若 $\text{Conflict}(s) > \tau_{\text{conflict}}$ 且 $C(s) > \tau_C$, 标记为 EXPERT_DEPENDENT
4. **全噪声状态 (Fully Noisy):** 若 $L(s) < \varepsilon_L$, 直接标记为 NOISY

3.6 与其他模块的协调

数据消费者	接收的数据	用途
模块 4 (估值)	$L(s), C(s), N(s), D(s)$	计算 $V_{\text{add}}(s)$ 和 $V_{\text{remove}}(s)$
模块 6 (压缩)	$D(s), \text{boundary_idx}$	决定保留/丢弃数据
模块 7 (主动学习)	classification, $L(s)$	选择动作和分配预算
模块 5 (路由)	ExpertConflict(s)	触发仲裁

3.7 计算复杂度

步骤	时间复杂度	空间复杂度
状态统计量	$O(N \cdot K)$	$O(K)$
一致性计算	$O(N \cdot M \cdot C_f + N \cdot K)$	$O(K)$
噪声分数	$O(N \cdot K)$	$O(N)$
冗余度	$O(N \cdot K)$	$O(K)$
边界检测	$O(N \cdot K \log K)$	$O(N)$

3.8 关键超参数

参数	符号	默认值	含义
误差阈值	τ_r	0.5 (归一化)	区分高/低误差状态
密度阈值	τ_ρ	0.05	区分高/低密度状态
一致性阈值	τ_C	0.3	区分一致/不一致状态
可学习性阈值	τ_L	0.4	区分可学习/不可学习状态
冗余度阈值	τ_D	0.7	区分充足/不足覆盖
SCX 差距阈值	$\tau_{\text{SCX_gap}}$	0.3	专家可靠性的最小显著差异
边界阈值	τ_{boundary}	0.3	边界样本检测灵敏度

参数	符号	默认值	含义
一致性组合权重	α, β, γ	(0.5, 0.3, 0.2)	标签/专家/密度一致性的权重

模块 4：状态数据估值 (State Data Valuation)

4.1 问题设定

数据估值的核心是回答两个问题：1. **采集价值** $V_{\text{add}}(s)$ ：如果现在从状态 s 获取更多标注样本，对模型性能的提升有多大？2. **压缩价值** $V_{\text{remove}}(s)$ ：如果移除状态 s 中的现有样本，模型性能损失有多大？

这两个价值是互补的，分别驱动主动学习和数据压缩。

4.2 数学模型

4.2.1 采集价值 (Acquisition Value) 基本定义：

$$V_{\text{add}}(s) = \bar{r}(s) \cdot \rho(s) \cdot L(s) \cdot [1 - D(s)] \cdot \max_m \text{SCX}_m(s)$$

因子分析：

因子	含义	直觉
$\bar{r}(s)$	当前误差	误差越大，改进空间越大
$\rho(s)$	状态概率	常见状态的影响更大
$L(s)$	可学习性	只有可学习状态才值得投入
$1 - D(s)$	覆盖不足	已覆盖足够则不再需要
$\max_m \text{SCX}_m(s)$	最好专家的可靠性	有可靠专家才能获取高质量标签

动态性： $V_{\text{add}}(s)$ 是动态的——当获取更多数据后， $\bar{r}(s)$ 下降， $D(s)$ 上升，价值自然衰减。

带成本的采集价值：

$$V_{\text{add}}^{(\text{cost})}(s) = V_{\text{add}}(s) - \lambda_c \cdot \text{Cost}_{\text{acquire}}(s)$$

其中 $\text{Cost}_{\text{acquire}}(s)$ 是从状态 s 获取标注的成本（如 DFT 计算时间）。

4.2.2 压缩价值 (Compression Value) 定义：

$$V_{\text{remove}}(s) = \bar{r}(s) \cdot D(s) \cdot (1 - \rho(s)) \cdot \frac{1}{1 + L(s)}$$

- 高 $\bar{r}(s)$ + 高 $D(s)$ ：模型在状态 s 上表现差但数据冗余 → 可以移除部分冗余
- 低 $\rho(s)$ ：低频状态的移除影响有限
- 低 $L(s)$ ：不可学习状态的移除没有损失

更精确的压缩价值:

使用 leave-one-state-out 评估:

$$V_{\text{remove}}(s) = \mathcal{L}(\theta; \mathcal{D} \setminus \mathcal{D}_s) - \mathcal{L}(\theta; \mathcal{D})$$

其中 $\mathcal{L}(\theta; \mathcal{D})$ 是模型在全部数据上的损失, $\mathcal{D}_s$ 是状态 s 的数据。

实际中不需要重训练, 可以用影响函数近似:

$$V_{\text{remove}}(s) \approx \frac{1}{|\mathcal{D}_s|} \sum_{x_i \in \mathcal{D}_s} \nabla_{\theta} \ell(x_i; \theta)^{\top} H_{\theta}^{-1} \nabla_{\theta} \mathcal{L}(\mathcal{D})$$

其中 H_{θ} 是 Hessian 矩阵。

4.2.3 与 Shapley 值的关系 Shapley 值 (样本 i 的边际贡献):

$$\phi_i = \sum_{S \subseteq \mathcal{D} \setminus \{i\}} \frac{|S|!(|\mathcal{D}| - |S| - 1)!}{|\mathcal{D}|!} [v(S \cup \{i\}) - v(S)]$$

其中 $v(S)$ 是在子集 S 上训练后的模型性能。

SCX 状态估值与 Shapley 的关系:

SCX 的状态估值是对 Shapley 值的状态级近似:

$$V_{\text{add}}(s) \approx \frac{1}{|\mathcal{D}_s|} \sum_{i \in \mathcal{D}_s} \phi_i$$

关键区别：- Shapley 值计算需要 $O(2^{|\mathcal{D}|})$ 次模型训练 $\rightarrow$ 计算不可行 - SCX 状态估值在 $O(K)$ 级别计算 $\rightarrow$ 可扩展到大型数据集 - SCX 通过状态结构引入先验知识（状态内样本共享价值）

4.2.4 与 Influence Function 的关系 影响函数（样本 i 对模型参数的影响）：

$$\mathcal{J}(x_i) = -H_{\theta}^{-1} \nabla_{\theta} \ell(x_i; \theta)$$

SCX 状态估值与影响函数的关系：

$$V_{\text{remove}}(s) \propto \frac{1}{|\mathcal{D}_s|} \sum_{i \in \mathcal{D}_s} \|\mathcal{J}(x_i)\|$$

即状态压缩价值近似等于状态内样本平均影响范数。

4.2.5 成本敏感的估值 标注成本：

$$\text{Cost}_{\text{acquire}}(s) = c_0 \cdot \mathbb{E}_{x \sim P(\cdot|s)} [\text{Cost}_{\text{label}}(x)]$$

对于 MLIP 场景， $\text{Cost}_{\text{label}}(x)$ 可能是 DFT 计算成本，与原子数、计算精度相关。

总体效用：

$$U_{\text{acquire}}(s) = V_{\text{add}}(s) - \lambda \cdot \text{Cost}_{\text{acquire}}(s)$$

4.3 输入输出规范

Input:

- r_bar: ndarray, shape=(K,), 状态平均误差 (模块3)
- rho: ndarray, shape=(K,), 状态概率 (模块1/3)
- L: ndarray, shape=(K,), 可学习性 (模块3)
- D: ndarray, shape=(K,), 冗余度 (模块3)
- SCX: ndarray, shape=(M, K), SCX可靠性 (模块2)
- cost_per_state: ndarray, shape=(K,), 可选, 标注成本
- lambda_cost: float, 成本敏感系数

Output:

- V_add: ndarray, shape=(K,), 采集价值
- V_remove: ndarray, shape=(K,), 压缩价值
- V_add_cost: ndarray, shape=(K,), 成本调整后的采集价值
- ranking_add: list[int], 按 V_add 降序排列的状态 ID
- ranking_remove: list[int], 按 V_remove 降序排列的状态 ID

4.4 实现方法

算法 4: 状态数据估值

Algorithm: ComputeStateValues(r_bar, rho, L, D, SCX)

Step 1: 采集价值

```

for s in 1..K:
    best_expert = max(SCX[:, s])
    V_add[s] = r_bar[s] * rho[s] * L[s] * (1 - D[s]) * best_expert

```

Step 2: 压缩价值

```
for s in 1..K:
    V_remove[s] = r_bar[s] * D[s] * (1 - rho[s]) / (1 + L[s] + eps)
```

Step 3: 成本调整 (若成本信息可用)

```
for s in 1..K:
    V_add_cost[s] = V_add[s] - lambda_cost * cost_per_state[s]
```

Step 4: 排序

```
ranking_add = argsort(V_add, descending)
ranking_remove = argsort(V_remove, descending)
```

Result: {V_add, V_remove, V_add_cost, ranking_add, ranking_remove}

4.5 与其他模块的协调

数据消费者	接收的数据	用途
模块 7 (主动学习)	$V_{\text{add}}(s)$, ranking_add	采集函数的输入
模块 6 (压缩)	$V_{\text{remove}}(s)$, ranking_remove	压缩优先级
人类可读报告	ranking 信息	可视化哪些状态最有价值

4.6 计算复杂度

步骤	时间复杂度	空间复杂度
采集价值	$O(K \cdot M)$	$O(K)$

步骤	时间复杂度	空间复杂度
压缩价值	$O(K)$	$O(K)$
排序	$O(K \log K)$	$O(K)$

4.7 关键超参数

参数	符号	默认值	含义
成本系数	λ	0.0	成本影响采集决策的权重
忽略噪声价值	-	true	若 $L(s) < \varepsilon_L$, 强制 $V_{\text{add}}(s) = 0$

模块 5：专家路由与仲裁 (Expert Routing & Arbitration)

5.1 问题设定

在多专家系统中，对于给定的输入 x ，我们需要决定：哪个（或哪些）专家最适合处理这个样本？路由策略分为硬路由（选一个专家）和软路由（加权组合多个专家）。当专家之间出现显著分歧时，还需要仲裁机制。

5.2 数学模型

5.2.1 硬路由 (Hard Routing) 路由决策：

$$m^*(x) = \arg \min_m \sum_{s=1}^K \gamma_s(x) \cdot \hat{R}_m(s)$$

即选择在 x 所属状态下期望风险最低的专家。

带计算成本的路由：

$$m^*(x) = \arg \min_m \left[\sum_{s=1}^K \gamma_s(x) \cdot \hat{R}_m(s) + \lambda \cdot C_m \right]$$

其中 C_m 是专家 m 的计算成本（推理时间、GPU 内存等）。

基于 SCX 置信度的路由：

$$m^*(x) = \arg \max_m \sum_{s=1}^K \gamma_s(x) \cdot \text{SCX}_m(s)$$

选择最可能给出可靠预测的专家。

5.2.2 软加权 (Soft Weighting) 基于风险的软权重：

$$w_m(x) = \frac{\exp(-\alpha \cdot \sum_s \gamma_s(x) \cdot \hat{R}_m(s))}{\sum_{n=1}^M \exp(-\alpha \cdot \sum_s \gamma_s(x) \cdot \hat{R}_n(s))}$$

其中 $\alpha \geq 0$ 是温度参数：- $\alpha \rightarrow 0$ ：退化为均匀加权 $w_m(x) \rightarrow 1/M$ - $\alpha \rightarrow \infty$ ：退化为硬路由（one-hot 权重）

基于 SCX 的软权重：

$$w_m(x) = \frac{\sum_s \gamma_s(x) \cdot \text{SCX}_m(s)^\beta}{\sum_n \sum_s \gamma_s(x) \cdot \text{SCX}_n(s)^\beta}$$

其中 $\beta \geq 1$ 控制权重的集中程度。

最终预测:

$$f_{\text{ensemble}}(x) = \sum_{m=1}^M w_m(x) \cdot f_m(x)$$

5.2.3 冲突检测 (Conflict Detection) 状态级冲突:

$$\text{Conflict}(s) = \frac{2}{M(M-1)} \sum_{m < n} \mathbb{E}_{x \sim P(\cdot|s)} [\ell(f_m(x), f_n(x))]$$

或者使用专家风险估计的方差:

$$\text{Conflict}(s) = \frac{1}{M} \sum_{m=1}^M (\hat{R}_m(s) - \bar{R}(s))^2, \quad \bar{R}(s) = \frac{1}{M} \sum_m \hat{R}_m(s)$$

逐样本冲突:

$$\text{Conflict}(x) = \frac{2}{M(M-1)} \sum_{m < n} \ell(f_m(x), f_n(x))$$

冲突阈值触发的动作:

$$\text{Action}(x) = \begin{cases} \text{route_to_expert} & \text{if } \exists m : \text{SCX}_m(s(x)) \gg \text{SCX}_{\text{others}}(s(x)) \\ \text{request_relabel} & \text{if } \text{Conflict}(x) > \tau_{\text{conflict}} \text{ and oracle_available} \\ \text{weighted_ensemble} & \text{otherwise} \end{cases}$$

5.2.4 仲裁策略 (Arbitration Strategies) 当冲突发生时, 有以下仲裁选项:

策略 A: 加权集成 (Weighted Ensemble)

使用 5.2.2 节的软权重。

策略 B: 最佳专家 (Best Expert)

$$f(x) = f_{m^*}(x), \quad m^* = \arg \min_m \sum_s \gamma_s(x) \cdot \hat{R}_m(s)$$

策略 C: 置信度过滤 (Confidence Filtering)

仅使用 $\text{SCX}_m(s(x)) > \tau_{\text{SCX}}$ 的专家:

$$\mathcal{M}_{\text{conf}}(x) = \{m \mid \sum_s \gamma_s(x) \cdot \text{SCX}_m(s) > \tau_{\text{SCX}}\}$$

$$f(x) = \frac{1}{|\mathcal{M}_{\text{conf}}(x)|} \sum_{m \in \mathcal{M}_{\text{conf}}(x)} f_m(x)$$

策略 D: 元专家 (Meta-Expert)

训练一个轻量级元模型 $g: \mathbb{R}^M \rightarrow \mathcal{Y}$, 以专家输出 $[f_1(x), \dots, f_M(x)]$ 为输入:

$$f(x) = g(f_1(x), f_2(x), \dots, f_M(x); \theta_g)$$

5.2.5 与 MoE 的关系和区别 传统 MoE (Mixture of Experts):

$$f_{\text{MoE}}(x) = \sum_{m=1}^M g_m(x; \theta_g) \cdot f_m(x)$$

其中 $g_m(x; \theta_g)$ 是一个学习得到的门控网络，通常通过 softmax 输出。

SCX 路由 vs. MoE 门控:

维度	MoE 门控	SCX 路由
训练方式	端到端梯度下降	基于专家风险的显式估计
所需数据	需要大量标注	可在无标签/少量标签下工作
可解释性	黑盒	可解释（因为风险是按状态估计的）
领域知识注入	困难	容易（通过状态定义）
动态性	固定权重（除非微调）	随专家风险估计更新
冷启动	需要预训练	可以直接使用先验知识

互补性：SCX 路由可以作为 MoE 门控的初始化或正则化约束。

5.3 输入输出规范

Input:

- x: ndarray, shape=(batch, d_X), 输入样本
- gamma_func: callable, x -> gamma, shape=(batch, K), 软分配
- R: ndarray, shape=(M, K), 状态条件专家风险（模块2）
- SCX: ndarray, shape=(M, K), SCX可靠性（模块2）
- experts: list[callable], M 个专家函数
- alpha: float, 软权重的温度参数
- strategy: str, 'hard'/'soft_weighted'/'confidence_filter'/'meta_expert'
- tau_SCX: float, 置信度过滤阈值
- lambda_cost: float, 计算成本系数
- costs: ndarray, shape=(M,), 专家计算成本

Output:

- y_pred: ndarray, shape=(batch,), 路由/融合后的预测
- weights: ndarray, shape=(batch, M), 专家权重
- selected_expert: ndarray, shape=(batch,), 硬路由选择的专家ID
- conflict_score: ndarray, shape=(batch,), 冲突分数
- routing_info: dict, 包含路由决策的元数据

For soft weighting:

- $w_m(x) \in [0,1]$, $\sum_m w_m(x) = 1$, 每个样本的专家权重分布

5.4 实现方法

算法 5: 专家路由与仲裁

Algorithm: RouteAndArbitrate(x, experts, R, SCX, gamma_func, strategy)

Step 1: 获取状态分配

```
gamma = gamma_func(x) # shape=(batch, K)
```

Step 2: 计算状态条件专家风险

```
for each expert m:
    weighted_R[m] = sum_s gamma * R[m, s] # shape=(batch,)
```

Step 3: 路由决策

```
if strategy == 'hard':
    selected = argmin_m weighted_R
    weights = one_hot(selected, M)

elif strategy == 'soft_weighted':
    weights = softmax(-alpha * weighted_R, axis=-1)
```

```

elif strategy == 'confidence_filter':
    weighted_SCX[m] = sum_s gamma * SCX[m, s]
    mask = weighted_SCX > tau_SCX
    if all(mask == False):
        mask = True # fallback: use all experts
    weights = mask / sum(mask)

```

Step 4: 冲突检测

```

predictions = stack([f_m(x) for f_m in experts]) # shape=(M, batch)
conflict = pairwise_disagreement(predictions)

```

Step 5: 生成最终预测

```

y_pred = sum_m weights[:, :, m] * predictions[m, :, :]

```

(可选) 若 $\text{conflict} > \text{tau_conflict}$ 且仲裁策略不同:

```

调用 arbitration_strategy(x, predictions, weights, conflict)

```

Result: {y_pred, weights, conflict}

5.5 与其他模块的协调

上游模块	提供的数据	用途
模块 1 (状态发现)	$\gamma_s(x)$	计算加权风险
模块 2 (可靠性估计)	$R_m(s)$, $\text{SCX}_m(s)$	路由的核心依据
模块 3 (四分类)	$\text{ExpertConflict}(s)$	触发仲裁

下游模块	提供的数据	用途
模块 7 (主动学习)	conflict_score	relabel 动作的 触发条件

5.6 计算复杂度

步骤	时间复杂度	空间复杂度
获取状态分配	$O(\text{batch} \cdot K \cdot d_\phi)$	$O(\text{batch} \cdot K)$
计算加权风险	$O(\text{batch} \cdot M \cdot K)$	$O(\text{batch} \cdot M)$
专家推理	$O(\text{batch} \cdot M \cdot C_f)$	$O(\text{batch} \cdot M)$
冲突检测	$O(\text{batch} \cdot M^2)$	$O(\text{batch} \cdot M)$

5.7 关键超参数

参数	符号	默认值	含义
温度参数	α	1.0	控制软权重的集中程度
SCX 置信度阈值	τ_{SCX}	0.7	置信度过滤的最小 SCX
冲突阈值	τ_{conflict}	0.3	触发仲裁的冲突级别
成本系数	λ	0.0	计算成本在路由中的权重
仲裁策略	-	weighted_ensemble	冲突时的仲裁方法

模块 6: SCX-Compress (冗余压缩)

6.1 问题设定

给定训练数据集 $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, SCX-Compress 的目标是找到一个**加权子集** $\mathcal{D}_c = \{(x_i, y_i, w_i)\}$, 使得: 1. **模型性能保真**: 在 $\mathcal{D}_c$ 上训练的性能接近在 $\mathcal{D}$ 上训练的性能 2. **状态结构保留**: 状态划分和专家可靠性估计在压缩前后的变化不超过容差 3. **压缩率最大化**: $|\mathcal{D}_c| \ll |\mathcal{D}|$

6.2 数学模型

6.2.1 冗余分数 (Redundancy Score) 逐样本冗余分数:

$$\mathcal{R}(x_i) = D(s_i) \cdot C(s_i) \cdot \left(1 - \max_m \text{SCX}_m(s_i)\right)$$

其中 $s_i = \arg \max_s \gamma_s(x_i)$ 。

分解分析: - 高 $D(s_i)$: 状态已被充分覆盖 $\rightarrow$ 数据可能冗余 - 高 $C(s_i)$: 状态内一致性高 $\rightarrow$ 样本信息不独特 - 低 $\max_m \text{SCX}_m(s_i)$: 没有专家特别可靠 $\rightarrow$ 不需要保留专家特定的样本

相对冗余 (考虑状态内变异):

$$\mathcal{R}_{\text{rel}}(x_i) = \frac{\mathcal{R}(x_i)}{\max_{x_j \in s_i} \mathcal{R}(x_j)}$$

6.2.2 加权 Coreset 构造 样本保留概率:

$$p_{\text{keep}}(x_i) = \min \left(1, \frac{n_{\text{keep}}(s_i) \cdot \text{Importance}(x_i)}{\sum_{x_j \in s_i} \text{Importance}(x_j)} \right)$$

其中：

$$\text{Importance}(x_i) = \frac{1}{1 + \mathcal{R}(x_i)}$$

$$n_{\text{keep}}(s) = \max \left(n_{\min}(s), |\mathcal{D} \cap s| \cdot [1 - D(s)] \cdot \frac{1}{1 + \bar{r}(s)} \right)$$

保留后样本权重：

$$w_i = \frac{1}{p_{\text{keep}}(x_i)} \quad (\text{逆概率加权})$$

或者更精细的权重（考虑状态内重要性）：

$$w_i = \frac{|\mathcal{D} \cap s_i|}{|\mathcal{D}_c \cap s_i|} \cdot \frac{\text{Importance}(x_i)}{\sum_{x_j \in s_i} \text{Importance}(x_j)}$$

6.2.3 保真约束 (Fidelity Constraints) 约束 1：状态级风险保真

$$\left| \hat{R}_m(s; \mathcal{D}_c) - \hat{R}_m(s; \mathcal{D}) \right| < \delta_R, \quad \forall m, s$$

其中 $\hat{R}_m(s; \mathcal{D}_c) = \frac{\sum_{i \in \mathcal{D}_c \cap s} w_i \cdot \ell(f_m(x_i), y_i)}{\sum_{i \in \mathcal{D}_c \cap s} w_i}$ 。

约束 2：状态概率保真

$$|\rho(s; \mathcal{D}_c) - \rho(s; \mathcal{D})| < \delta_\rho, \quad \forall s$$

其中 $\rho(s; \mathcal{D}_c) = \frac{\sum_{i \in \mathcal{D}_c} \gamma_s(x_i) \cdot w_i}{\sum_{i \in \mathcal{D}_c} w_i}$ 。

约束 3：模型性能保真

$$|\mathcal{L}(\theta_c; \mathcal{D}_{\text{val}}) - \mathcal{L}(\theta; \mathcal{D}_{\text{val}})| < \delta_{\mathcal{L}}$$

其中 θ_c 是在 $\mathcal{D}_c$ 上训练的参数， θ 是在 $\mathcal{D}$ 上训练的参数。

6.2.4 边界 Anchor 保留策略 边界样本由于定义状态边界，对整体系统至关重要，必须保留：

$$\mathcal{B} = \{x_i \mid \text{BoundaryScore}(x_i) > \tau_{\text{boundary}}\}$$

$$\mathcal{D}_c \supseteq \mathcal{B} \quad (\text{强制约束})$$

边界样本权重：

$$w_i = 1 + \text{BoundaryScore}(x_i) \quad \text{对边界样本额外加权}$$

边界覆盖要求：

$$\frac{|\mathcal{D}_c \cap \mathcal{B}|}{|\mathcal{B}|} = 1 \quad (\text{所有边界样本必须保留})$$

6.2.5 与 Coreset Selection 的关系 传统 Coreset: 选择最小加权重子集, 使得在全数据上的损失近似等于在 coreset 上的加权损失:

$$\left| \sum_{i \in \mathcal{D}} \ell(x_i; \theta) - \sum_{i \in \mathcal{D}_c} w_i \ell(x_i; \theta) \right| < \varepsilon, \quad \forall \theta \in \Theta$$

SCX-Compress 区别于传统 Coreset:

维度	传统 Coreset	SCX-Compress
选择准则	几何/覆盖/误差上界	状态冗余 + 专家可靠性
保真目标	损失函数上界	状态结构 + 专家风险 + 模型性能
权重	基于覆盖计数的均匀权重	基于状态重要性的加权
边界处理	无	强制保留边界 anchor
可解释性	低 (黑盒选择)	高 (按状态解释为什么保留/丢弃)

6.2.6 与 Dataset Distillation 的关系 Dataset Distillation: 合成少量伪样本 $\tilde{\mathcal{D}}$ 使得:

$$\theta(\tilde{\mathcal{D}}) \approx \theta(\mathcal{D}), \quad \text{where } \theta(\mathcal{D}) = \arg \min_{\theta} \mathcal{L}(\mathcal{D}; \theta)$$

关系: - SCX-Compress 是**选择型**压缩 (select existing data points) - Dataset distillation 是**合成型**压缩 (generate new synthetic data points) - 两者可以**互补**: 先用 SCX-Compress 选择最核心的样本, 再对这些样本进行蒸馏 - SCX-Compress 的计算成本低于蒸馏, 更适合科学计算场景

6.3 输入输出规范

Input:

- X: ndarray, shape=(N, d_X), 输入数据
- y: ndarray, shape=(N,), 标签
- gamma: ndarray, shape=(N, K), 软分配 (模块1)
- R: ndarray, shape=(M, K), 专家风险 (模块2)
- SCX: ndarray, shape=(M, K), SCX可靠性 (模块2)
- C: ndarray, shape=(K,), 一致性 (模块3)
- D: ndarray, shape=(K,), 冗余度 (模块3)
- r_bar: ndarray, shape=(K,), 状态平均误差 (模块3)
- boundary_idx: list[int], 边界样本索引 (模块3)
- compression_rate: float, 目标压缩率 (默认 0.5)
- delta_R: float, 风险保真容差 (默认 0.05)
- delta_rho: float, 概率保真容差 (默认 0.02)

Output:

- X_compressed: ndarray, shape=(N_c, d_X), 压缩后的数据
- y_compressed: ndarray, shape=(N_c,), 压缩后的标签
- weights: ndarray, shape=(N_c,), 样本权重
- keep_mask: ndarray, shape=(N,), bool 保留掩码
- compression_stats: dict, 压缩统计:
 - compression_ratio_achieved: float
 - max_R_deviation: float
 - max_rho_deviation: float
 - boundary_preserved: bool

6.4 实现方法

算法 6: SCX-Compress

Algorithm: SCXCompress(X, y, gamma, R, SCX, C, D, r_bar, boundary_idx, rate)

Step 1: 计算冗余分数

```
for i in 1..N:
    s_i = argmax_s gamma[i, :]
    R_i = D[s_i] * C[s_i] * (1 - max(SCX[:, s_i]))
    importance[i] = 1 / (1 + R_i)
```

Step 2: 确定状态级保留量

```
for s in 1..K:
    n_s = count_members(X, gamma, s)
    n_keep[s] = max(
        max(n_boundary_in_s, 5),          # 至少保留边界样本 + 最小样本数
        n_s * (1 - D[s]) * (1 / (1 + r_bar[s] + eps))
    )
```

Step 3: 选择保留样本

```
keep_mask = boundary_idx # 强制保留边界样本
for s in 1..K:
    candidates = members_of_state(s) \ boundary_idx
    n_to_keep_in_state = n_keep[s] - count_boundary_in_state(s)
    prob = importance[candidates] / sum(importance[candidates])
    selected = sample_without_replacement(candidates, n_to_keep_in_state, prob)
    keep_mask[selected] = True
```

Step 4: 计算逆概率权重

```
for s in 1..K:
    n_total_s = count_members(X, gamma, s)
    n_keep_s = sum(keep_mask & members_of_state(s))
    for i in members_of_state(s):
```

```

    if keep_mask[i]:
        weights[i] = n_total_s / n_keep_s * importance[i]

```

Step 5: 保真检验

```

R_deviation = check_R_fidelity(R, X_compressed, y_compressed, weights, gamma)
rho_deviation = check_rho_fidelity(rho, X_compressed, weights, gamma)

while max(R_deviation) > delta_R or max(rho_deviation) > delta_rho:
    # 对偏差最大的状态追加样本
    s_violate = argmax(R_deviation)
    add_more_samples(s_violate, n_additional=5)
    recompute_weights()
    recompute_deviations()

```

Result: {X_compressed, y_compressed, weights, keep_mask, stats}

6.5 与其他模块的协调

上游模块	接收的数据	用途
模块 1 (状态发现)	$\gamma_s(x)$, 状态中心	状态成员资格和边界定义
模块 2 (可靠性估计)	$R_m(s)$, $SCX_m(s)$	专家可靠性的保真约束
模块 3 (四分类)	$C(s)$, $D(s)$, $\bar{r}(s)$, boundary_idx	冗余分数计算
模块 4 (估值)	$V_{\text{remove}}(s)$	压缩优先级排序

6.6 计算复杂度

步骤	时间复杂度	空间复杂度
冗余分数计算	$O(N \cdot (K + M))$	$O(N)$
状态级保留量	$O(K)$	$O(K)$
加权采样	$O(N \log N)$	$O(N)$
保真检验	$O(N_c \cdot M \cdot C_f)$	$O(M \cdot K)$
权重计算	$O(N)$	$O(N)$

6.7 关键超参数

参数	符号	默认值	含义
目标压缩率	-	0.5	保留原始数据量的比例
风险保真容差	δ_R	0.05	压缩前后专家风险的最大允许偏差
概率保真容差	δ_ρ	0.02	压缩前后状态概率的最大允许偏差
边界阈值	τ_{boundary}	0.3	边界判定的灵敏度
最小每状态保留数	$n_{\min}$	5	每个状态至少保留的样本数
保真迭代追加量	-	5	保真违规时每次追加的样本数

模块 7：主动学习策略 (Active Learning Policy)

7.1 问题设定

主动学习的核心问题：在有限的标注预算 B 下，选择哪些样本进行标注，以及选择何种标注方式，能最大化模型性能提升。SCX 将这个决策从逐样本 (point-wise) 升级为逐状态 (state-wise)，并且拓展了动作空间——不只是“标注哪个”，还包括“如何标注”。

7.2 数学模型

7.2.1 动作空间 (Action Space) SCX 定义了五种动作，每种动作对应不同的数据操作：

$$\mathcal{A} = \{\text{acquire}, \text{relabel}, \text{downweight}, \text{discard}, \text{route}\}$$

动作	含义	预算消耗	效果
acquire	从状态 s 获取新标注样本	高（如 DFT 计算时间）	增加训练数据量
relabel	对状态 s 中已有样本重新标注（找更强 oracle）	中（专家时间）	修正错误标签

动作	含义	预算消耗	效果
downweight	降低状态 s 中疑似噪声样本的权重	低（计算操作）	减少噪声影响
discard	丢弃状态 s 中的极端噪声样本	低（删除操作）	消除有害数据
route	将状态 s 的标注任务路由到最可靠的专家	中（推理成本）	提高标签质量

7.2.2 效用函数 (Utility Function) 动作-状态对的效用:

$$U(a, s) = \underbrace{\Delta \mathcal{L}(a, s)}_{\text{期望性能提升}} - \lambda_b \cdot \underbrace{\text{Cost}(a, s)}_{\text{预算消耗}}$$

各动作的效用分解:

acquire 的效用:

$$U(\text{acquire}, s) = V_{\text{add}}(s) - \lambda_b \cdot \text{Cost}_{\text{label}}(s)$$

其中 $V_{\text{add}}(s)$ 来自模块 4, $\text{Cost}_{\text{label}}(s)$ 是在状态 s 中标注一个样本的成本。

relabel 的效用:

$$U(\text{relabel}, s) = \bar{r}(s) \cdot \text{Conflict}(s) \cdot \rho(s) - \lambda_b \cdot \text{Cost}_{\text{relabel}}$$

高误差 + 高冲突 $\rightarrow$ 重新标注可能分歧解决的收益大。

downweight 的效用:

$$U(\text{downweight}, s) = N(s) \cdot \rho(s) \cdot \bar{r}(s) - \lambda_b \cdot \text{Cost}_{\text{downweight}}$$

高噪声 + 高频率 $\rightarrow$ 降权的收益大。

discard 的效用:

$$U(\text{discard}, s) = N(s) \cdot (1 - \rho(s)) \cdot \bar{r}(s) - \lambda_b \cdot \text{Cost}_{\text{discard}}$$

高噪声 + 低频 $\rightarrow$ 丢弃无害。

route 的效用:

$$U(\text{route}, s) = \max_m \text{SCX}_m(s) \cdot \rho(s) \cdot \bar{r}(s) - \lambda_b \cdot \text{Cost}_{\text{route}}(m^*)$$

路由到最可靠专家，适用于 $\text{Conflict}(s)$ 高但 SCX 差异大的状态。

7.2.3 预算分配策略 贪心策略 (Greedy):

每一步选择 $(a^*, s^*) = \arg \max_{a,s} U(a, s)$ ，重复直到预算耗尽。

比例分配 (Proportional):

$$B(s) = B \cdot \frac{V_{\text{add}}(s)}{\sum_t V_{\text{add}}(t)}$$

然后在每个状态 s 内，按 $U(a, s)$ 分配该状态的子预算给各动作。

Epsilon-贪心探索:

以概率 ε 选择次优状态（探索新状态），以概率 $1 - \varepsilon$ 贪心选择。

多臂赌博机视角:

将每个 (s, a) 对看作一个臂，应用 Thompson sampling 或 UCB:

$$a^*(s) = \arg \max_a \left[\hat{U}(a, s) + \kappa \cdot \sqrt{\frac{2 \log t}{n_{a,s}}} \right]$$

其中 t 是总轮数， $n_{a,s}$ 是 (a, s) 被选中的次数。

7.2.4 从 Point-wise 到 State-wise 的升级 传统 Point-wise AL:

$$x^* = \arg \max_{x \in \mathcal{U}} a_{\text{score}}(x, \mathcal{D}_L)$$

其中 a_{score} 是采集函数（如不确定性、多样性）。

SCX State-wise AL:

$$\text{Step 1 (选择状态)} : s^* = \arg \max_s \max_a U(a, s)$$

$$\text{Step 2 (选择动作)} : a^* = \arg \max_a U(a, s^*)$$

$$\text{Step 3 (选择样本)} : \begin{cases} x^* = \arg \max_{x \in \mathcal{U} \cap s^*} \text{distance}(x, \mathcal{D}_L \cap s^*) & \text{if } a^* = \text{acquire} \\ x^* = \arg \max_{x \in \mathcal{D}_L \cap s^*} \text{Conflict}(x) & \text{if } a^* = \text{relabel} \end{cases}$$

为什么要 State-wise:

1. **统计效率:** 状态级聚合减少了估计方差（借用”同状态样本共享信息”）

2. 计算效率：状态数 $K \ll N$ ，优化在 $O(K)$ 空间而非 $O(N)$
3. 语义意义：状态提供可解释的决策单元
4. 多样性：避免只关注不确定性最高的离群点

7.2.5 与 DP-GEN 的对比 DP-GEN (Deep Potential Generator):

exploration : $x_{\text{new}} = \arg \max_x \sigma_{\text{model}}^2(x)$ (模型方差)

exploitation : 筛选低能量结构加入训练集

对比:

维度	DP-GEN	SCX-AL
选择准则	模型方差	$V_{\text{add}}(s)$ 综合价值
状态意识	隐式（通过结构筛选）	显式（状态是核心概念）
动作空间	仅标注	acquire/relabel/downweight/discard/route
噪声处理	无专门机制	通过 NoiseScore 明确识别
专家利用	无	通过 $R_m(s)$ 和 $\text{SCX}_m(s)$
预算分配	无	显式状态级预算分配

7.2.6 与 D-optimality 的对比 D-optimality: 选择设计矩阵 X 使得 $\det(X^\top X)$ 最大，最小化参数估计的置信椭圆体积。

对比: - D-optimality 只关注线性模型参数估计的方差 - SCX-AL 关注非线性模型的泛化性能 - D-optimality 不考虑噪声和专家可靠性 - D-optimality 是 SCX 中覆盖度 $D(s)$ 的相关概念

7.2.7 与 DIRECT 的对比 DIRECT (Diverse and Relevant Example seleCTion): 选择既多样又相关的样本。

对比: - DIRECT: $x^* = \arg \max [\lambda \cdot \text{diversity}(x) + (1 - \lambda) \cdot \text{relevance}(x)]$ -
 SCX-AL: $s^* = \arg \max_s V_{\text{add}}(s)$, 然后在 s^* 内选多样性样本 - DIRECT 是
 point-wise, SCX-AL 是 two-level (state + point) - DIRECT 的 relevance
 取决于特定任务, SCX-AL 的 V_{add} 是通用的

7.3 输入输出规范

Input:

- X_labeled: ndarray, shape=(N_L, d_X), 已标注数据
- y_labeled: ndarray, shape=(N_L,), 已标注标签
- X_unlabeled: ndarray, shape=(N_U, d_X), 未标注数据
- gamma_labeled: ndarray, shape=(N_L, K), 已标注数据的软分配
- gamma_unlabeled: ndarray, shape=(N_U, K), 未标注数据的软分配
- V_add: ndarray, shape=(K,), 采集价值 (模块4)
- R: ndarray, shape=(M, K), 专家风险 (模块2)
- SCX: ndarray, shape=(M, K), SCX可靠性 (模块2)
- r_bar: ndarray, shape=(K,), 状态平均误差 (模块3)
- rho: ndarray, shape=(K,), 状态概率 (模块1/3)
- C: ndarray, shape=(K,), 一致性 (模块3)
- N: ndarray, shape=(K,), 噪声分数 (模块3)
- D: ndarray, shape=(K,), 冗余度 (模块3)
- classification: dict, 状态四分类 (模块3)
- budget: float, 总标注预算
- costs: dict, 各动作的成本
- strategy: str, 'greedy'/'proportional'/'epsilon_greedy'/'thompson'
- epsilon: float, 探索概率 (默认 0.1)

Output:

- selected_actions: list[tuple(state_id, action, budget_spent)]

- `X_new`: `ndarray`, `shape=(B_acquire, d_X)`, 新采集样本
- `y_new`: `ndarray`, `shape=(B_acquire,)`, 新标注标签
- `relabel_indices`: `list[int]`, 需要重新标注的样本索引
- `downweight_indices`: `list[int]`, 需要降权的样本索引
- `discard_indices`: `list[int]`, 需要丢弃的样本索引
- `route_decisions`: `dict[sample_idx -> expert_id]`, 路由决策
- `remaining_budget`: `float`, 剩余预算

7.4 实现方法

算法 7: SCX 主动学习策略

Algorithm: `SCXActiveLearning(X_U, state_info, experts, budget, strategy)`

初始化: `budget_remaining = budget`

`while budget_remaining > 0:`

 Step 1: 更新状态信息 (来自模块1-4)

 Step 2: 对每个状态和动作计算效用 $U(a, s)$

 Step 3: 选择 (s^*, a^*)

 if `strategy == 'greedy'`:

$(s^*, a^*) = \operatorname{argmax}_{\{a,s\}} U(a, s)$

 elif `strategy == 'epsilon_greedy'`:

 if `random() < epsilon`:

$(s^*, a^*) = \text{random}(s, a) \text{ from non-zero utility}$

 else:

$(s^*, a^*) = \operatorname{argmax}_{\{a,s\}} U(a, s)$

```

elif strategy == 'thompson':
    (s*, a*) = sample from posterior over U(a, s)

Step 4: 执行动作
cost = costs[a*]
if cost > budget_remaining:
    continue # 跳过, 选下一个

if a* == 'acquire':
    candidates = X_U intersect state s*
    x_new = select_diverse_representatives(candidates, n=1)
    y_new = query_oracle(x_new) # 调用 oracle
    X_L = X_L ∪ {(x_new, y_new)}
    X_U = X_U \ {x_new}
elif a* == 'relabel':
    candidates = X_L intersect state s*
    x_relabel = argmax Conflict(x) for x in candidates
    y_corrected = query_strong_oracle(x_relabel)
    update_label(x_relabel, y_corrected)
elif a* == 'downweight':
    weight_reduction = C[s*] * (1 - N[s*])
    for x in X_L intersect state s*:
        sample_weight[x] *= weight_reduction
elif a* == 'discard':
    for x in X_L intersect state s*:
        if NoiseScore(x) > tau_noise_extreme:
            X_L = X_L \ {x}
elif a* == 'route':
    for x in X_L intersect state s*:

```

```

m_best = argmax_m SCX[m, s*]
label_replacement = f_{m_best}(x)
# 使用专家标注作为伪标签

budget_remaining -= cost
update_state_estimates() # 更新 r_bar, rho, D, L 等

```

Result: {selected_actions, X_L, sample_weights, budget_remaining}

7.5 主动学习的停止条件

1. 预算耗尽: $\text{budget_remaining} \leq 0$
2. 边际效用为零: $\max_{a,s} U(a, s) \leq 0$ (所有动作都不值得)
3. 性能达标: 验证集性能达到目标阈值
4. 状态饱和: 所有状态的 $D(s) > \tau_{D,\text{stop}}$ 或 $L(s) < \tau_{L,\text{stop}}$

7.6 与其他模块的协调

上游模块	接收的数据	用途
模块 1 (状态发现)	$\gamma_s(x), \rho(s)$	状态范围划分
模块 2 (可靠性估计)	$R_m(s), \text{SCX}_m(s)$	路由动作和 relabel 条件
模块 3 (四分类)	classification, $L(s), N(s)$	动作选择
模块 4 (估值)	$V_{\text{add}}(s), V_{\text{remove}}(s)$	采集和压缩的效 用核心

输出方向	发送的数据	用途
→ 模块 2	新标注数据	更新专家风险估计
→ 模块 1	新数据	更新状态划分

7.7 计算复杂度

步骤	时间复杂度	空间复杂度
效用计算	$O(K \cdot M + K)$	$O(K \cdot \mathcal{A})$
动作选择	$O(K \cdot \mathcal{A})$	$O(K \cdot \mathcal{A})$
执行 acquire	$O(N_U \cdot K)$ (选代表)	$O(N_U)$
状态信息更新	$O(N_L \cdot M \cdot C_f)$	$O(M \cdot K)$

7.8 关键超参数

参数	符号	默认值	含义
探索率	ε	0.1	epsilon-贪心中的探索概率
预算成本系数	λ_b	1.0	效用中成本项的权重
停止冗余阈值	$\tau_{D,\text{stop}}$	0.9	所有状态超此值则停止
停止可学习阈值	$\tau_{L,\text{stop}}$	0.05	所有状态低于此值则停止
Thomson 探索指数	κ	0.5	UCB 探索系数

模块 8：与 Paper 1-3 的连接

8.1 问题设定

SCX（论文 4）不是凭空出现的，而是对前三篇论文经验的数学抽象和理论升华。理解这四篇论文的层级关系，对于理解 SCX 的理论动机和实际应用至关重要。

8.2 四篇论文的层级关系

论文层级（从具体到抽象）：

Paper 1: ACE/PACE Expert Gauge + Merging

具体方法：势函数合并

Paper 2: Residual-State Error Maps

具体工具：误差可视化与归因

Paper 3: Expert Compiler + Distillation

工程框架：多专家管理平台

Paper 4: SCX (State-Conditioned eXpertise)

数学理论：数据价值与专家引导学习

8.3 Paper 1: Gauge-Normalized Expert Merging → SCX 的 Expert Reliability 基础

8.3.1 Paper 1 的核心内容 Paper 1 处理多个 ACE (Atomic Cluster Expansion) 势函数的合并问题。核心挑战：- 不同 ACE 势函数使用不同的规

范 (gauge), 不能直接线性组合 - 需要找到规范变换, 使得势函数在”物理等价”的意义上对齐

数学形式:

$$f_m(x) = \sum_i c_{m,i} \cdot B_i(x) \quad (\text{ACE 展开})$$

$$\text{Gauge transformation: } \tilde{c}_m = G \cdot c_m$$

其中 G 是规范变换矩阵, 使得 $\tilde{c}_m$ 在规范下对齐。

8.3.2 SCX 从 Paper 1 继承的洞察

1. **专家不是等价的:** 不同势函数在不同结构区域表现不同 $\rightarrow R_m(s)$ 的动机
2. **全局指标不够:** 势函数整体的 RMSE 掩盖了局部行为差异 $\rightarrow$ 命题 1 的动机
3. **规范差异:** 专家之间存在系统偏差 $\rightarrow$ SCX 中的冲突检测
4. **合并不是唯一方法:** 与其合并专家, 不如基于状态选择专家 $\rightarrow$ 路由的动机

8.3.3 形式化连接 Paper 1 中规范对齐后的残差:

$$r_m(x) = |f_m^{\text{align}}(x) - f^*(x)|$$

Paper 1 隐式地使用结构类型 (如 sp^2 , sp^3 , 断键) 作为状态, 这直接启发了 SCX 的状态定义:

Paper 1 的状态: $s(x) \in \{\text{sp}^2, \text{sp}^3, \text{broken bond}, \text{surface}, \dots\}$

SCX 的状态: $s: \mathcal{X} \rightarrow \{1, \dots, K\}$ (一般化)

Paper 1 中的专家特定状态误差矩阵:

$$E_{m,s} = \frac{1}{|\mathcal{D}_s|} \sum_{x_i \in \mathcal{D}_s} \ell(f_m(x_i), f^*(x_i))$$

正是 SCX 中 $R_m(s)$ 的前身。

8.4 Paper 2: Residual-State Error Maps $\rightarrow$ SCX 的 State Discovery

8.4.1 Paper 2 的核心内容 Paper 2 提出了残差状态图 (Residual-State Maps): 将势函数的预测误差映射到结构状态空间, 以便直观识别哪些结构区域是某个势函数的薄弱环节。

核心方法: 1. 对每个样本 x_i , 计算残差 $r_i = \ell(f(x_i), y_i)$ 2. 对每个结构状态 s , 计算 $\bar{r}(s) = \mathbb{E}[r_i | x_i \in s]$ 3. 绘制 $\bar{r}(s)$ 的热力图

8.4.2 SCX 从 Paper 2 继承的洞察

1. 状态空间的可视化: 通过残差热力图发现模式 $\rightarrow$ 状态发现模块
2. 高误差 高价值: Paper 2 显示有些高误差状态是”不可救药的” $\rightarrow$ 命题 2
3. 状态边界的过渡区域: 观察到边界附近误差最高 $\rightarrow$ 边界检测
4. 误差归因: 将误差归因到具体结构状态 $\rightarrow V_{\text{add}}(s)$ 中的 $r(s)$

8.4.3 形式化连接 Paper 2 的残差状态图定义:

$$\mathcal{R}(s) = \{(m, \bar{r}_m(s)) \mid m = 1, \dots, M\}$$

SCX 将其推广为:

$$R_m(s) = \mathbb{E}_{x \sim P(\cdot|s)}[\ell(f_m(x), f^*(x))]$$

Paper 2 中残差图降维选择的 PCA/UMAP 状态空间直接对应 SCX 的模块 1。

8.5 Paper 3: Expert Compiler Distillation $\rightarrow$ SCX 的 Expert Routing + Teacher Generation

8.5.1 Paper 3 的核心内容 Paper 3 构建了一个多专家势函数平台 (Expert Compiler)，能够：1. 管理多个势函数专家 2. 根据输入结构自动选择/组合专家 3. 将多专家知识蒸馏到单一学生模型

8.5.2 SCX 从 Paper 3 继承的洞察

1. 路由基础设施：Paper 3 实现了初步的路由机制 $\rightarrow$ 模块 5
2. 蒸馏作为通道：将专家知识传递给轻量模型 $\rightarrow$ 软路由的逻辑
3. 动态专家组合：根据输入特征调整专家权重 $\rightarrow w_m(x)$ 的动机
4. 处理异质专家：不同架构、不同训练数据的专家 $\rightarrow$ 通用框架的必要性

8.5.3 形式化连接 Paper 3 的路由机制可以视为 SCX 路由的特定实例化：

$$\text{Paper 3 路由: } m^*(x) = \arg \min_m \sum_s \gamma_s(x) \cdot \underbrace{\text{validation_err}(m, s)}_{\text{对应 } R_m(s)}$$

Paper 3 的蒸馏损失：

$$\mathcal{L}_{\text{distill}}(x) = \sum_m w_m(x) \cdot \ell(f_{\text{student}}(x), f_m(x))$$

其中 $w_m(x)$ 由专家在该状态下的表现决定 $\rightarrow$ 正是 SCX 的软路由公式。

8.6 四篇论文的统一框架

8.6.1 概念映射

SCX 概念	Paper 1 实例	Paper 2 实例	Paper 3 实例
专家 f_m	ACE 势函数 $f_1, f_2, \dots$	被评估的势函数	编译器管理的势函数池
状态 s	化学环境类型	残差图中的结构聚类	结构特征区间
$R_m(s)$	势函数在特定环境下 RMSE	残差状态图的值	验证集上每专家每结构误差
$\text{SCX}_m(s)$	势函数在环境下 $<$ 阈值概率	(隐式) 可接受残差区域	专家在结构上的置信度
$V_{\text{add}}(s)$	增加某环境训练数据的价值	(隐式) 高残差 + 高密度状态	需要补充训练的区域
路由 $m^*(x)$	(被合并取代)	(被分析取代)	专家编译器自动选择
动作 $\mathcal{A}$	采集新 DFT 数据	分析误差模式	路由 + 蒸馏 + 采集

8.6.2 论文的递进关系

Paper 1: "我注意到不同的 ACE 势函数在不同结构上表现不同"

↓ 具体化

Paper 2: "让我系统地把误差映射到结构状态空间"

↓ 系统化

Paper 3: "让我建造一个能自动管理多个势函数的平台"

↓ 理论化

Paper 4: "这个问题的本质是：状态条件专家可靠性 + 数据价值"

8.6.3 在科学 ML 中的统一叙事 统一目标：从多专家系统中学习，提高数据效率和模型可靠性。

方法论演化： 1. **Paper 1:** 用规范变换合并专家（消除分歧） 2. **Paper 2:** 可视化专家分歧的分布（理解分歧） 3. **Paper 3:** 构建系统管理分歧（利用分歧） 4. **Paper 4 (SCX):** 将分歧视为信息，用数学框架统一数据价值和专家路由（理论化分歧）

技术传承：

Paper 1 中规范对齐的专家集合

→ 状态条件误差矩阵 $E_{\{m,s\}}$

→ Paper 2 的残差状态图

→ 状态空间发现与降维

→ Paper 3 的路由和蒸馏

→ 软加权和仲裁

→ Paper 4 的 SCX 统一框架

8.7 实际应用价值

这种层级关系的实际意义：

1. **理论指导实践：** SCX 为 Paper 3 的编译器提供优化目标（最大化 V_{add} ）

- 2. 实践验证理论: Paper 1-3 的实验数据为 SCX 的命题提供实证
- 3. 跨领域泛化: SCX 的抽象使得方法可以迁移到 MLIP 之外的领域 (医学图像、NLP 等)
- 4. 持续迭代: Paper 3 的编译器可作为 SCX 的实验平台, SCX 的理论可指导编译器的改进

8.8 计算复杂度

论文	主要计算成本	与 SCX 的关系
Paper 1	$O(N \cdot C_{ACE} \cdot M)$	提供初始 $R_m(s)$ 估计
Paper 2	$O(N \cdot (d^2 + K \cdot M))$	提供状态空间基础
Paper 3	$O(N \cdot M \cdot C_f + N_{\text{distill}} \cdot C_{\text{student}})$	提供路由实验平台
Paper 4	$O(N \cdot M \cdot C_f + N \cdot K \cdot M)$	理论统一, 计算开销最小

完整系统架构图

SCX: State-Conditioned eXpertise
完整系统架构 — 数据流与控制流

Raw Data X
(原子结构/图像/
文本/传感器)

模块 1：状态发现与表示 (State Discovery & Representation)

表示映射	降维	聚类	软分配 $\pi_s(x)$
$: X \rightarrow$	PCA/UMAP	GMM/	$(s), \pi_s, \Sigma_s$
ACE/SOAP	$\rightarrow$	K-means	边界分数 $B(x)$

模块 2：专家可靠性估计 $\pi_s(x) \quad (s)$
(Expert Reliability Estimation)

有标签	$R_m(s)$ 估计	模块 3：数据四分类
场景	(加权平均)	(Data Four-
		Classification)

无标签	Disagreement	$C(s) \quad N(s) \quad D(s)$
场景	近似 $R_m(s)$	

小样本
场景

贝叶斯/层次
风险估计

$$L(s) = C \cdot (1-N)$$

Output: $R_m(s), SCX_m(s)$
(shape: $M \times K$ 矩阵)

Valuable/

Redundant/
Noisy/
Expert-dependent

四分类决策:

模块 5: 专家路由与仲裁
(Expert Routing)

硬路由
 $m^*(x)$

软加权
 $w_m(x)$

冲突检测
Conflict

仲裁策略
集成/过滤

Output: $y_{pred}, w_m(x)$

模块 4: 状态估值
(State Data
Valuation)

$$V_{add}(s) = \bar{r} \cdot \cdot L \cdot (1-D) \cdot \max_m SCX_m(s)$$

$$V_{remove}(s) = \bar{r} \cdot D \cdot (1-)$$

模块 6: SCX-Compress (压缩)

冗余分数 加权 Coreset
 $R(x_i)$ 构造+保真检验

Output: D_c , weights, keep_mask

模块 7: 主动学习策略 (Active Learning Policy)

效用函数 $U(a, s)$ 计算

$U(\text{acquire})$ $U(\text{relabel})$ $U(\text{downweight})$ $U(\text{discard})$ $U(\text{route})$

动作选择策略 (Greedy / Thompson / Epsilon-greedy)

预算分配 $\rightarrow (s^*, a^*) \rightarrow$ 执行 $\rightarrow$ 更新

输出:

- 新增标注样本 (acquire)
- 重新标注索引 (relabel)

- 降权/丢弃索引 (downweight/discard)
- 路由决策 (route-to-expert)

模型更新 (Model Update)

重训练 / 增量训练 / 微调

循环 (Loop to Module 1)

用新数据更新状态和专家估计

与 Paper 1-3 的连接 (模块 8)

Paper 1	Paper 2	Paper 3	Paper 4 (SCX)
专家规范	→ 残差状态	→ 专家编译	→ 状态条件理论基础
合并	映射	器蒸馏	(本框架)

概念映射: $f_m \rightarrow \text{ACE}$; $s \rightarrow \text{化学环境}$; $_s(x) \rightarrow \text{结构类型分配}$

$R_m(s) \rightarrow \text{势函数每结构RMSE}$; $V_{\text{add}} \rightarrow \text{采集DFT数据价值}$

数据流摘要

X (原始数据)

[模块1] 状态发现

$_s(x)$, (s) , 边界分数

[模块3] 数据四分类

$$C(s), N(s), D(s), L$$

[模块4] 数据估值

 $V_{\text{add}}(s), V_{\text{remove}}(s)$

[模块2] 专家可靠性

$$R_m(s), SCX_m(s)$$

[模块5] 路由 对已有样本做预测/标注

[模块7] 主动学习 选择 (s^*, a^*) 执行动作

[模块6] 压缩 (可选)

模型更新

[回到 \[模块1\] \(循环\)](#)

总结

SCX 框架的核心贡献可以用一句话概括：

专家的可靠性不是全局常数，而是状态条件的；数据的价值不是固有属性，而是相对于当前模型、状态空间和可用专家的条件量。

八个子模块构成了一个完整的闭环系统：1. **状态发现**将异质输入空间分解为语义一致的状态 2. **可靠性估计**量化每个专家在每个状态的表现 3. **数据分类**将状态分为四类，驱动不同的决策 4. **数据估值**给出每个状态的采集和压缩价值 5. **路由仲裁**基于状态条件可靠性做专家选择 6. **压缩**在保真约束下移除冗余数据 7. **主动学习**在状态级做预算分配和动作选择 8. **论文连接**将这四篇工作的层次关系和传承路径说清楚

SCX 的价值在于它不是一个拼凑的工具箱，而是一个**数学上自洽的框架**——每个模块的输出是下一个模块的输入，所有公式共享一致的状态空间基础，最终形成一个可解释、可扩展的主动学习 + 专家管理闭环。